List Interface

01 March 2025 20:41

Introduction

List is a child interface of collection interface. If we want to represent a group of individual object as a single entity, where **duplicates are allowed** and **insertion order** must be **preserved(W.R.T index)**, then we should go for **List** interface.

We can preserve insertion order with index and we can differentiate duplicate objects by using index. Hence, index will play very important role in **List** interface.

Collection (I)
↑
List (I)

Detailed Explanation

public interface List<E> extends Collection<E>

An ordered collection (also known as a *sequence*). The user of this interface has precise control over where in the list each element is inserted. The user can access elements by their integer index (position in the list), and search for elements in the list.

Unlike sets, lists typically allow duplicate elements. More formally, lists typically allow pairs of elements e1 and e2 such that e1.equals(e2), and they typically allow multiple null elements if they allow null elements at all. It is not inconceivable that someone might wish to implement a list that prohibits duplicates, by throwing runtime exceptions when the user attempts to insert them, but we expect this usage to be rare.

The List interface places additional stipulations, beyond those specified in the Collection interface, on the contracts of the iterator, add, remove, equals, and hashCode methods. Declarations for other inherited methods are also included here for convenience.

The List interface provides four methods for positional (indexed) access to list elements. Lists (like Java arrays) are zero based. Note that these operations may execute in time proportional to the index value for some implementations (the LinkedList class, for example). Thus, iterating over the elements in a list is typically preferable to indexing through it if the caller does not know the implementation.

The List interface provides a special iterator, called a ListIterator, that allows element insertion and replacement, and bidirectional access in addition to the normal operations that the Iterator interface provides. A method is provided to obtain a list iterator that starts at a specified position in the list.

The List interface provides two methods to search for a specified object. From a performance standpoint, these methods should be used with caution. In many implementations they will perform costly linear searches.

The List interface provides two methods to efficiently insert and remove multiple elements at an arbitrary point in the list.

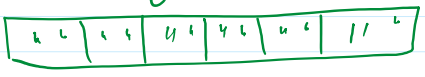
Note: While it is permissible for lists to contain themselves as elements, extreme caution is advised: the equals and hashCode methods are no longer well defined on such a list.

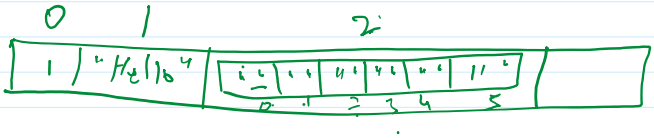
Some list implementations have restrictions on the elements that they may contain. For example, some implementations prohibit null elements, and some have restrictions on the types of their elements. Attempting to add an ineligible element throws an unchecked exception, typically NullPointerException or ClassCastException. Attempting to query the presence of an ineligible element may throw an exception, or it may simply return false; some implementations will exhibit the former behavior and some will exhibit the latter. More generally, attempting an operation on an ineligible element whose completion would not result in the insertion of an ineligible element into the list may throw an exception or it may succeed, at the option of the implementation. Such exceptions are marked as "optional" in the specification for this interface.

This interface is a member of the Java Collections Framework.

From <https://docs.oracle.com/javase/8/docs/api/java/util/List.html>

List of strings





List interface defines the following specific methods:

Method Summary

All Methods | Instance Methods | Abstract Methods | Default Methods

Modifier and Type	Method and Description
boolean	add(E e) Appends the specified element to the end of this list (optional operation).
void	add(int index, E element) Inserts the specified element at the specified position in this list (optional operation).
boolean	addAll(Collection<? extends E> c) Appends all of the elements in the specified collection to the end of this list, in the order that they are returned by the specified collection's iterator (optional operation).
boolean	addAll(int index, Collection<? extends E> c) Inserts all of the elements in the specified collection into this list at the specified position (optional operation).
void	clear() Removes all of the elements from this list (optional operation).
boolean	contains(Object o) Returns true if this list contains the specified element.
boolean	containsAll(Collection<?> c) Returns true if this list contains all of the elements of the specified collection.
boolean	equals(Object o) Compares the specified object with this list for equality.
E	get(int index) Returns the element at the specified position in this list.
int	indexOf(Object o) Returns the index of the first occurrence of the specified element in this list, or -1 if this list does not contain the element.
boolean	isEmpty() Returns true if this list contains no elements.
Iterator<E>	iterator() Returns an iterator over the elements in this list in proper sequence.
int	lastIndexOf(Object o) Returns the index of the last occurrence of the specified element in this list, or -1 if this list does not contain the element.
ListIterator<E>	listIterator() Returns a list iterator over the elements in this list (in proper sequence).
ListIterator<E>	listIterator(int index) Returns a list iterator over the elements in this list (in proper sequence), starting at the specified position in the list.
E	remove(int index) Removes the element at the specified position in this list (optional operation).
boolean	remove(Object o) Removes the first occurrence of the specified element from this list, if it is present (optional operation).
boolean	removeAll(Collection<?> c) Removes from this list all of its elements that are contained in the specified collection (optional operation).
default void	replaceAll(UnaryOperator<E> operator) Replaces each element of this list with the result of applying the operator to that element.
boolean	retainAll(Collection<?> c) Retains only the elements in this list that are contained in the specified collection (optional operation).
E	set(int index, E element) Replaces the element at the specified position in this list with the specified element (optional operation).
int	size() Returns the number of elements in this list.

default void	sort(Comparator<? super E> c) Sorts this list according to the order induced by the specified Comparator.
default Spliterator<E>	spliterator() Creates a Spliterator over the elements in this list.
List<E>	subList(int fromIndex, int toIndex) Returns a view of the portion of this list between the specified fromIndex, inclusive, and toIndex, exclusive.
Object[]	toArray() Returns an array containing all of the elements in this list in proper sequence (from first to last element).
<T> T[]	toArray(T[] a) Returns an array containing all of the elements in this list in proper sequence (from first to last element); the runtime type of the returned array is that of the specified array.

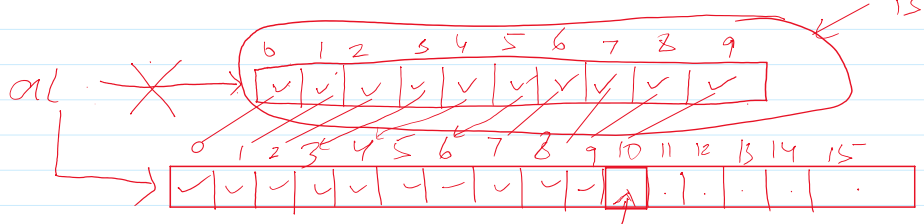
ArrayList

- **The underlying data structure is resizable array or growable array.**
- **Duplicates are allowed.**
- **Insertion order is preserved.**
- **Heterogeneous objects are allowed (except TreeSet and TreeMap everywhere heterogeneous objects are allowed).**
- **'null' insertion is allowed.**

Constructors

1. **ArrayList al = new ArrayList(); ==> Creates an empty ArrayList object with default initial capacity 10. Once array list reaches its max capacity, then a new array list object will be created with:**

   newCapacity = (current_capacity * 3/2) + 1 ← interview Question.



$= \left(\frac{5}{10} * \frac{3}{2}\right) + 1$

$= 16$

$\left(\frac{8}{16} \times \frac{3}{2}\right) + 1$

$= 25.$

2. **ArrayList al = new ArrayList(int initialCapacity); ==> Creates an empty analyst object with specified initial capacity.**
3. **ArrayList al = new ArrayList(Collection c); ==> Creates an equivalent array list object for the given collection.**

Constructors

Constructor and Description
ArrayList() Constructs an empty list with an initial capacity of ten.
ArrayList(Collection<? extends E> c) Constructs a list containing the elements of the specified collection, in the order they are returned by the collection's iterator.
ArrayList(int initialCapacity) Constructs an empty list with the specified initial capacity.

All Methods | Instance Methods | Concrete Methods

Modifier and Type	Method and Description
boolean	add(E e) Appends the specified element to the end of this list.
void	add(int index, E element) Inserts the specified element at the specified position in this list.
boolean	addAll(Collection<? extends E> c) Appends all of the elements in the specified collection to the end of this list, in the order that they are returned by the specified collection's Iterator.
boolean	addAll(int index, Collection<? extends E> c) Inserts all of the elements in the specified collection into this list, starting at the specified position.
void	clear() Removes all of the elements from this list.
Object	clone() Returns a shallow copy of this ArrayList instance.
boolean	contains(Object o) Returns true if this list contains the specified element.
void	ensureCapacity(int minCapacity) Increases the capacity of this ArrayList instance, if necessary, to ensure that it can hold at least the number of elements specified by the minimum capacity argument.
void	forEach(Consumer<? super E> action) Performs the given action for each element of the Iterable until all elements have been processed or the action throws an exception.
E	get(int index) Returns the element at the specified position in this list.
int	indexOf(Object o) Returns the index of the first occurrence of the specified element in this list, or -1 if this list does not contain the element.
boolean	isEmpty() Returns true if this list contains no elements.
Iterator<E>	iterator() Returns an iterator over the elements in this list in proper sequence.
int	lastIndexOf(Object o) Returns the index of the last occurrence of the specified element in this list, or -1 if this list does not contain the element.
ListIterator<E>	listIterator() Returns a list iterator over the elements in this list (in proper sequence).
ListIterator<E>	listIterator(int index) Returns a list iterator over the elements in this list (in proper sequence), starting at the specified position in the list.
E	remove(int index) Removes the element at the specified position in this list.
boolean	remove(Object o) Removes the first occurrence of the specified element from this list, if it is present.
boolean	removeAll(Collection<?> c) Removes from this list all of its elements that are contained in the specified collection.
boolean	removeIf(Predicate<? super E> filter) Removes all of the elements of this collection that satisfy the given predicate.
protected void	removeRange(int fromIndex, int toIndex) Removes from this list all of the elements whose index is between fromIndex, inclusive, and toIndex, exclusive.
void	replaceAll(UnaryOperator<E> operator) Replaces each element of this list with the result of applying the operator to that element.
boolean	retainAll(Collection<?> c) Retains only the elements in this list that are contained in the specified collection.
E	set(int index, E element) Replaces the element at the specified position in this list with the specified element.

int	size() Returns the number of elements in this list.
void	sort(Comparator<? super E> c) Sorts this list according to the order induced by the specified Comparator.
Spliterator<E>	spliterator() Creates a ***late-binding*** and *fail-fast* Spliterator over the elements in this list.
List<E>	subList(int fromIndex, int toIndex) Returns a view of the portion of this list between the specified fromIndex, inclusive, and toIndex, exclusive.
Object[]	toArray() Returns an array containing all of the elements in this list in proper sequence (from first to last element).
<T> T[]	toArray(T[] a) Returns an array containing all of the elements in this list in proper sequence (from first to last element); the runtime type of the returned array is that of the specified array.
void	trimToSize() Trims the capacity of this ArrayList instance to be the list's current size.

```java
import java.util.ArrayList;
import java.util.Random;
public class ArrayListThree {
    public static void main(String[] args) {
        Random r = new Random();
        ArrayList al = new ArrayList<>(20);
        int i;
        for(i = 0; i < 20; i++){
            al.add(r.nextInt(10, 99));
        }
        System.out.println("ArrayList: "+al);
        Integer x = (Integer)al.remove(5);
        System.out.println("Removed Element: " + x);
        System.out.println("After removal ArrayList: "+al);
        al.add("Shiv");
        System.out.println("ArrayList: "+al);
        System.out.println("\nRemoving object: "+al.remove("Shiv"));
        System.out.println("ArrayList: "+al);
        System.out.println("\nRemoving object: "+al.remove("Shiv"));
    }
}
```

Output:

```
ArrayList: [84, 46, 14, 60, 72, 60, 22, 44, 88, 46, 18, 95, 19, 35, 10, 38, 79, 30, 11, 65]
Removed Element: 60
After removal ArrayList: [84, 46, 14, 60, 72, 22, 44, 88, 46, 18, 95, 19, 35, 10, 38, 79, 30, 11, 65]
ArrayList: [84, 46, 14, 60, 72, 22, 44, 88, 46, 18, 95, 19, 35, 10, 38, 79, 30, 11, 65, Shiv]

Removing object: true
ArrayList: [84, 46, 14, 60, 72, 22, 44, 88, 46, 18, 95, 19, 35, 10, 38, 79, 30, 11, 65]

Removing object: false
```